

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ  
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ**

Студент гр. 4381

\_\_\_\_\_

Н.П. Имховик

Руководитель

\_\_\_\_\_

Т.Р. Жангиров

Санкт-Петербург

**2026**

## ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Н.П. Имховик

Группа: 4381

Тема практики: Генетические алгоритмы

Задание на практику:

Реализация генетического алгоритма для решения оптимизационной задачи.

Дано  $N$  матриц  $A$  с произвольными размерами ( $A_1: p_0 \times p_1, A_2: p_1 \times p_2, \dots, A_N: p_{N-1} \times p_N$ ). Необходимо определить порядок перемножения матриц, чтобы минимизировать количество операций умножения для вычисления результирующей матрицы  $B: p_0 \times p_N$ .

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 00.00.0000

Дата защиты отчета: 00.00.0000

|              |       |               |
|--------------|-------|---------------|
| Студент      | _____ | Н.П.Имховик   |
| Руководитель | _____ | Т.Р. Жангиров |

## **АННОТАЦИЯ**

В данном отчете представлена разработка и реализация генетического алгоритма для решения задачи оптимизации порядка перемножения цепочки матриц. В работе рассмотрены теоретические основы генетических алгоритмов, включая принципы селекции, скрещивания и мутации. Разработана структура классов, реализующая основные компоненты алгоритма: хромосому, популяцию, функцию приспособленности и генетические операторы. Создано графическое приложение на фреймворке Qt. Отчет содержит описание архитектуры программы, результатов экспериментов и выводов о применимости генетического подхода к решению задач оптимизации.

## **SUMMARY**

This report presents the development and implementation of a genetic algorithm for solving the matrix chain multiplication order optimization problem. The work covers the theoretical foundations of genetic algorithms, including the principles of selection, crossover, and mutation. A class structure implementing the main algorithm components has been developed, including chromosome, population, fitness function, and genetic operators. A graphical user interface has been created using the Qt framework. The report describes the program architecture, experimental results, and conclusions on the applicability of the genetic approach to solving optimization problems.

## СОДЕРЖАНИЕ

|                                             |    |
|---------------------------------------------|----|
| ВВЕДЕНИЕ.....                               | 5  |
| 1 ПОСТАНОВКА ЗАДАЧИ.....                    | 6  |
| 1.1 Предметная область.....                 | 6  |
| 1.2 Задачи практики.....                    | 6  |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ.....                    | 7  |
| 2.1. Формализация (кодирование) задачи..... | 7  |
| 2.2. Выбор функции приспособленности.....   | 7  |
| 2.3. Выбор методов отбора.....              | 8  |
| 2.4. Выбор методов скрещивания.....         | 8  |
| 2.5. Выбор методов мутации.....             | 9  |
| 2.6. Написание основного алгоритма.....     | 9  |
| 2.7. Написание GUI.....                     | 10 |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....              | 12 |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ.....                   | 13 |
| ЗАКЛЮЧЕНИЕ.....                             | 14 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....       | 15 |
| ПРИЛОЖЕНИЕ А.....                           | 16 |

## **ВВЕДЕНИЕ**

Учебная практика посвящена решению оптимизационных задач, а именно — задаче о порядке перемножения матриц. Классическое решение данной задачи достигается за время  $O(N^3)$  методами динамического программирования, однако с ростом размерности задачи больший интерес представляют эвристические алгоритмы. В рамках учебной практики, в частности, изучаются генетические алгоритмы, основанные на идеях дарвиновской эволюции.

Цель практики состоит в разработке и программной реализации генетического алгоритма для решения задачи о порядке перемножения матриц, а также написании пользовательского графического интерфейса для анализа эффективности алгоритма.

# **1 ПОСТАНОВКА ЗАДАЧИ**

## **1.1 Предметная область**

Генетические алгоритмы — эвристические алгоритмы поиска и оптимизации, основанные на дарвиновской теории эволюции. В ходе алгоритма происходит случайный подбор и комбинирование искоемых параметров с использованием механизмов эволюции: наследование, мутация, отбор.

## **1.2 Задачи практики**

- Изучение методов генетических алгоритмов
- Формализация (кодирование) задачи
- Выбор функции приспособленности
- Выбор методов отбора
- Выбор методов скрещивания
- Выбор методов мутации
- Написание основного алгоритма
- Написание GUI

## 2 РЕЗУЛЬТАТЫ РАБОТЫ

Для достижения поставленной цели был решен ряд задач.

### 2.1. Формализация (кодирование) задачи

Первым этапом реализации генетического алгоритма является формализация задачи — кодирование решения в виде вектора генов, где каждый ген может быть битом, числом или неким другим объектом. Обычно это вектор фиксированной длины. [1] В задаче о порядке перемножения матриц интерес представляют не сами матрицы, а их произведения, поэтому кодировать следует именно их.

Пусть дана цепочка из  $N$  матриц, тогда существует  $N-1$  позиция для умножения. Хромосома представляет собой вектор целых чисел, где каждый  $i$ -ый ген определяет приоритет умножения в позиции  $i$ . Любая случайная комбинация чисел в векторе задает однозначный, допустимый порядок перемножений.

### 2.2. Выбор функции приспособленности

Так как задача состоит в минимизации количества операций умножения для вычисления результирующей матрицы, целесообразно в качестве функции приспособленности использовать непосредственно количество арифметических операций, требуемых для выполнения заданного порядка перемножений. Поскольку ставится задача минимизации, меньшему значению функции должно соответствовать оптимальное решение.

Алгоритм вычисления функции приспособленности:

- Для каждой  $i$ -ой позиции произведения ставится в соответствие  $i$ -ый приоритет.
- Позиции сортируются по возрастанию приоритета, в результате чего получаем последовательность выполнения умножений  $S$ .
- Для каждого  $s_k$  из  $S$  определяются индексы левой и правой матриц, между которыми выполняется умножение в  $s_k$ , и их размеры.
- Пусть размеры заданы как  $[a,b]$  и  $[b,c]$ , тогда стоимость данного умножения  $a*b*c$  скалярных операций.

- Полученное значение добавляется к накопителю.
- Матрицы, для которых найдено произведение объединяются в одну результирующую размера  $[a, c]$ , а текущий массив матриц уменьшается на один элемент.
- Алгоритм продолжается до тех пор, пока в текущем массиве не останется одна матрица. Значение в накопителе — значение функции приспособленности для данного генома.

### **2.3. Выбор методов отбора**

Процесс отбора определяет, какие геномы будут оставлены для создания потомков, образующих следующее поколение. Рассматривались следующие методы отбора [1-2].

Правило рулетки: вероятность выбора каждой особи пропорциональна значению ее приспособленности. Для задачи минимизации требуется выполнение предварительного преобразования, чтобы меньшим значениям соответствовали большие вероятности. Недостатком является чувствительность к масштабу, если один геном окажется сильно более приспособленным, чем остальные то следующее поколение будет состоять из его потомков, что снизит разнообразие.

Ранжированный отбор: геномы сортируются по значению функции приспособленности, каждому из них присваиваются ранги. Далее геномы для скрещивания выбираются по рангам, что позволяет снизить влияние особей с слишком низким значением функции приспособленности.

Турнирный отбор: из популяции случайным образом выбирается  $n$  особей и среди них определяется особь с наименьшим значением функции приспособленности, данный процесс повторяется пока не будет сформировано следующее поколение. Данный метод был выбран в качестве основного, так как он не имеет проблем с масштабом, не требует глобального знания о всей популяции и прост в реализации.

### **2.4. Выбор методов скрещивания**



В данной работе для решения задачи применяется кодирование приоритетами, то есть каждый ген привязан к конкретной позиции умножения и представляет его приоритет. Но при этом решения содержащие повторы являются допустимыми, если принять, что операции с равными приоритетами выполняются в порядке возрастания индексов. Поэтому в качестве метода скрещивания подойдут одноточечное, двухточечное, равномерное. В работе представлены все три варианта для сравнения их эффективности.

## **2.5. Выбор методов мутации**

Так как решение кодируется целыми числами, в качестве методов мутации рассматриваются мутация обменом, обращением и перетасовкой.

## **2.6. Написание основного алгоритма**

Структура классов программы:

- **Chromosome** - хранит вектор целочисленных генов-приоритетов и значение функции приспособленности, обеспечивает декодирование генов в порядок умножения матриц через сортировку индексов по возрастанию приоритета, управляет доступом к генам и сбросом вычисленной приспособленности.
- **Population** - контейнер для хранения совокупности хромосом, предоставляет методы добавления и удаления особей, доступа к отдельным индивидам, поиска оптимального решения и проверки на пустоту.
- **FitnessFunction** - вычисляет приспособленность хромосомы путем декодирования порядка умножения и подсчета общего количества скалярных операций на основе размерностей матриц, возвращая целочисленное значение стоимости.
- **MatrixMultProblem** - хранит вектор размерностей матриц и количество матриц, обеспечивает валидацию данных и предоставление размерностей для вычисления фитнеса.

- GeneticAlgorithm - основной управляющий класс, содержит популяцию, параметры алгоритма и операторы селекции, скрещивания и мутации, управляет процессом эволюции: инициализацией популяции, вычислением функции приспособленности, формированием новых поколений через элитизм, отбор родителей, скрещивание и мутацию, отслеживанием истории решений.
- Операторы реализованы через абстрактные интерфейсы ISelection, ICrossover и IMutation. ISelection представлен турнирной селекцией с настраиваемым размером турнира. ICrossover включает одноточечное, двухточечное и равномерное скрещивание. IMutation включает мутацию обменом, обращением и перетасовкой. Все операторы работают с хромосомами через общий интерфейс.

## 2.7. Написание GUI

Структура пользовательского графического интерфейса:

Графический интерфейс построен на фреймворке Qt и состоит из главного окна MainWindow, которое содержит все элементы управления и отображения. Левая панель включает группы для ввода данных, настройки параметров алгоритма, управления выполнением и отображения текущей информации. Правая панель содержит график сходимости и таблицу популяции (Прил. А рис. 1).

Ввод данных осуществляется через текстовое поле для ручного ввода размерностей, а также кнопки загрузки из файла (Прил. А рис. 2) и генерации случайных данных. Параметры алгоритма настраиваются через счетчики и поля ввода: размер популяции, количество поколений, вероятности скрещивания и мутации, размер турнира, количество элитных особей. Выбор операторов выполняется через выпадающие списки для селекции, скрещивания и мутации.

Управление выполнением реализовано кнопками Start, Step, Finish и Back. Кнопка Start запускает непрерывную оптимизацию до достижения лимита поколений или остановки пользователем, Step выполняет одну

итерацию алгоритма, Finish доводит выполнение до конца, Back позволяет вернуться к предыдущему состоянию. Слайдер Speed регулирует задержку между поколениями при автоматическом выполнении.

Информационная панель отображает номер текущего поколения, наименьшее значение функции приспособленности, а также декодированный порядок умножения для оптимального решения. График сходимости визуализирует динамику изменения оптимального значения функции приспособленности по поколениям. Таблица показывает детальную информацию о каждой хромосоме в популяции: номер, набор генов и приспособленность (Прил. А рис. 3).

Для работы с длительными вычислениями используется отдельный поток AlgorithmWorker, который выполняет эволюцию в фоновом режиме, периодически обновляя состояние интерфейса через сигналы, что предотвращает зависание основного окна. История состояний позволяет перемещаться назад по шагам эволюции для анализа процесса.

### **3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ**

Основной язык реализации — C++. Фреймворк Qt — использован для разработки графического интерфейса, включая виджеты управления, отображение таблиц и графиков, загрузку файлов. Библиотека QtCharts - применена для визуализации сходимости алгоритма путем построения графиков изменения лучшего и среднего значения фитнеса по поколениям. Система сборки — Cmake. Среда разработки - VsCode.

## 4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша\_оценка>.

Отзыв руководителя с оценкой (см. рис. 1).

### Отзыв

#### о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок 1 — Отзыв руководителя

## **ЗАКЛЮЧЕНИЕ**

В ходе выполнения работы был разработан и реализован генетический алгоритм для решения задачи оптимизации порядка перемножения цепочки матриц [3]. Все поставленные задачи успешно решены.

В рамках работы был реализован полный набор генетических операторов, включающий турнирную селекцию, три варианта скрещивания (одноточечный, двухточечный и равномерный) и три варианта мутации (обменом, обращением и перетасовкой).

Разработанный графический интерфейс на базе фреймворка Qt обеспечивает взаимодействие с алгоритмом: загрузку и генерацию данных, настройку параметров, визуализацию процесса сходимости в реальном времени, пошаговое выполнение и просмотр истории состояний. Таблица популяции и график сходимости позволяют наглядно отслеживать динамику улучшения решений.

Таким образом, цель работы — разработка и программная реализация генетического алгоритма для задачи оптимизации порядка перемножения матриц с графическим интерфейсом для визуализации процесса - достигнута. Разработанное программное обеспечение может быть использовано для решения практических задач оптимизации, а также служить основой для дальнейших исследований в области применения генетических алгоритмов.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Панченко Т.В. Генетические алгоритмы: учебно-методическое пособие / под ред. Ю.Ю. Тарасевича. – Астрахань: Издательский дом «Астраханский университет», 2007. – [88].
2. Генетические алгоритмы на Python. – М.: ДМК Пресс, [2020]. – ISBN 978-5-97060-857-9.
3. Исходный код: <https://github.com/zuruxk/EducationalPractice.git>

## ПРИЛОЖЕНИЕ А

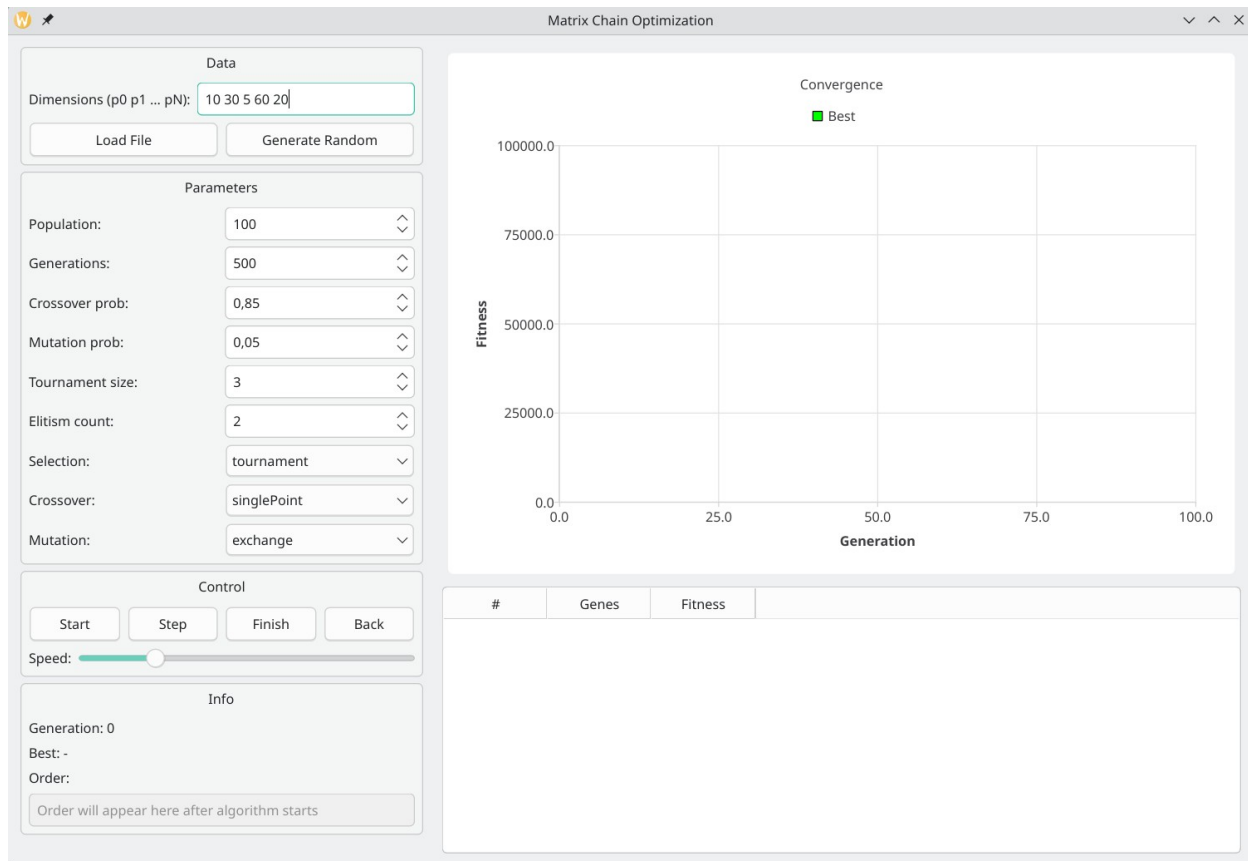


Рисунок 1 – Графический интерфейс программы



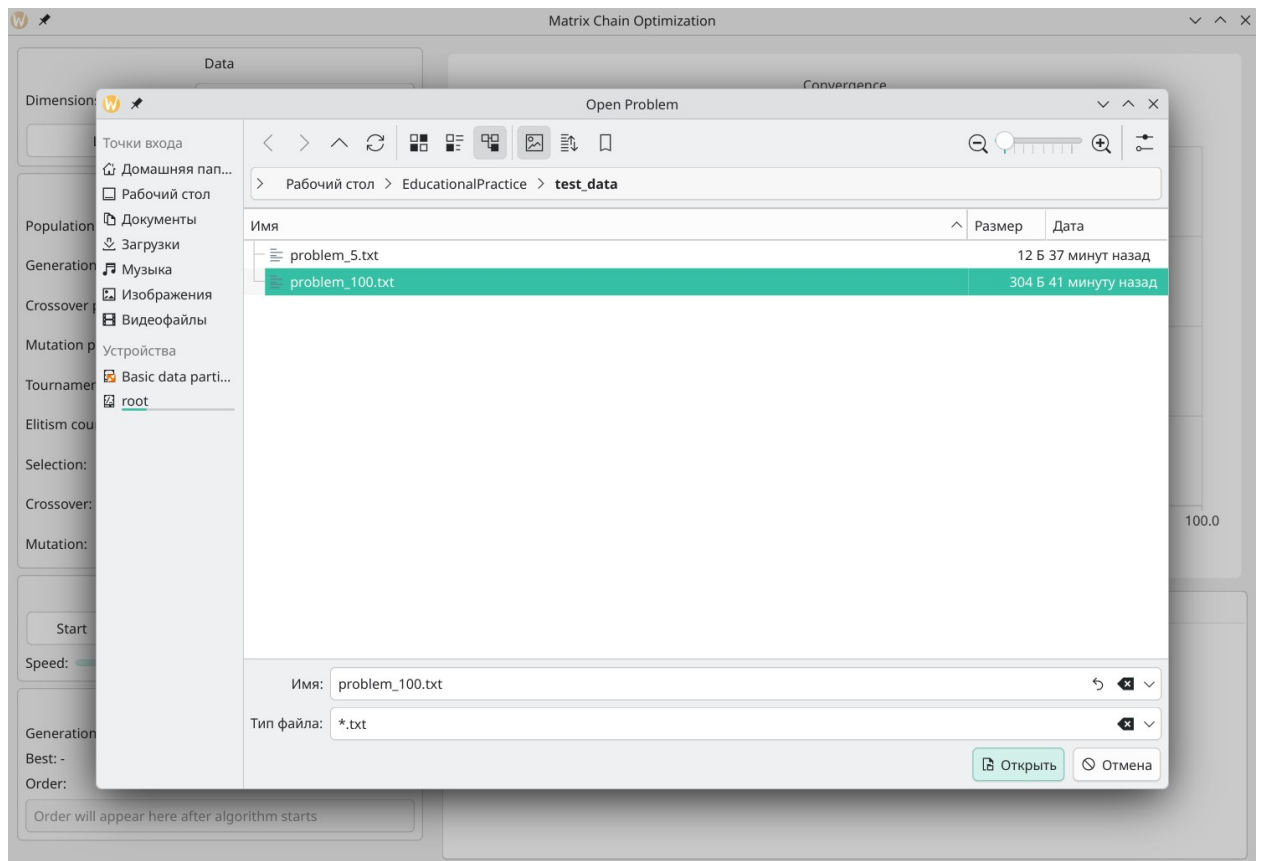


Рисунок 2 – Загрузка данных из файла

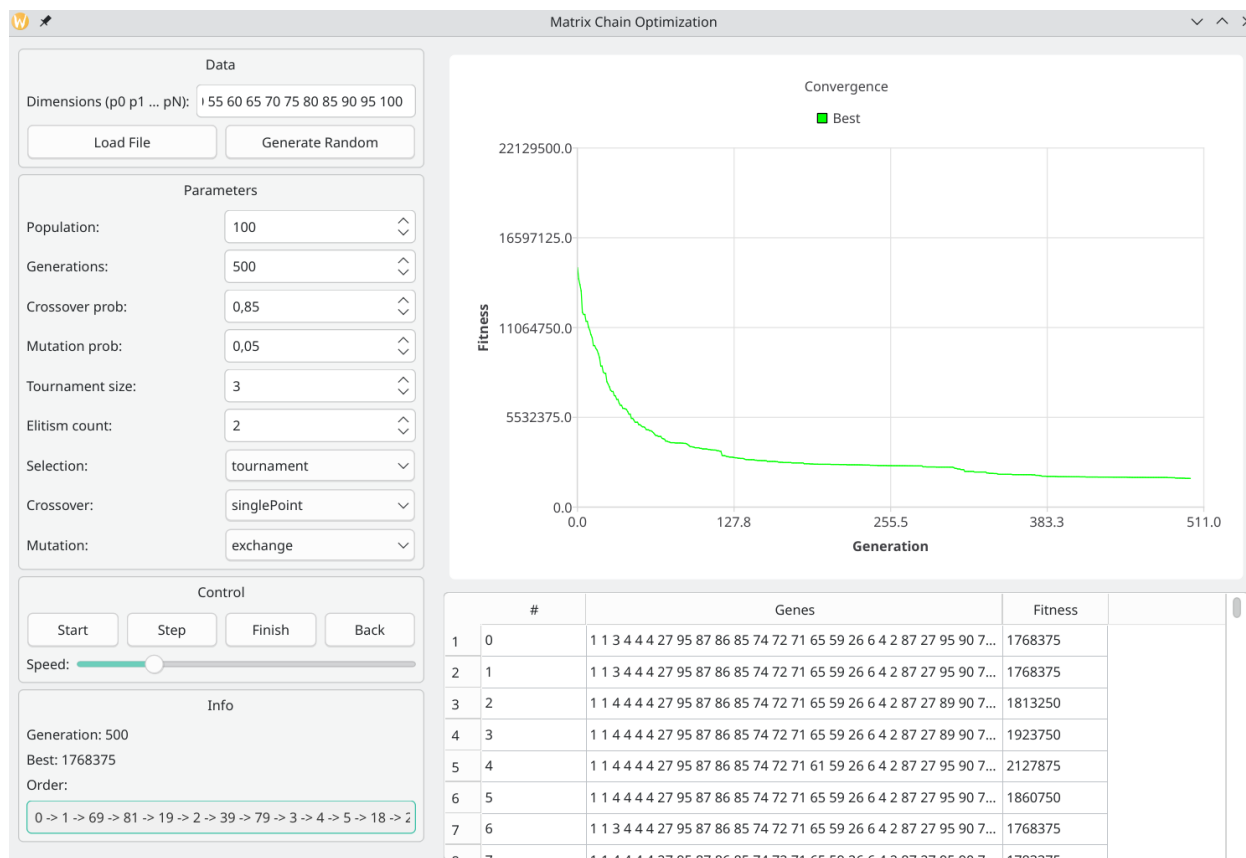


Рисунок 3 – Результат работы программы